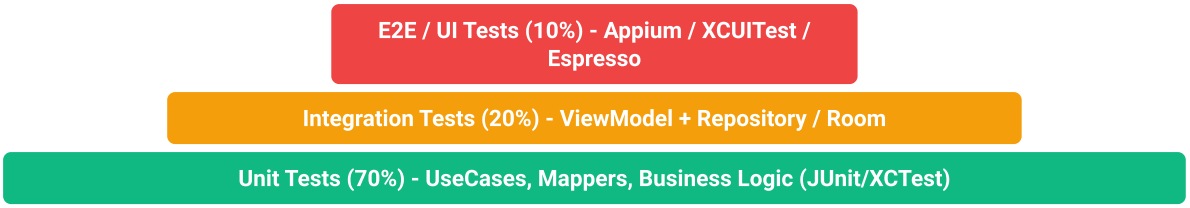
Estratégias de Testes Mobile

Pirâmide de Testes, Mocks, Stubs e Fakes em Prática

Ebook Técnico

1. PIRÂMIDE DE TESTES MOBILE



2. DEFINIÇÕES DOS TEST DOUBLES (GERARD MESZAROS)

Tipo	Propósito	Quando Utilizar
Dummy	Objeto passado apenas para preencher parâmetros obrigatórios. Nunca é invocado.	Instanciação de construtores complexos em testes unitários simples.
Stub	Fornece respostas enlatadas (hardcoded) para chamadas feitas durante o teste.	Simular respostas de sucesso ou erro de uma API de rede de forma rápida.
Spy	Envolve um objeto real ou Stub gravando informações sobre como foi chamado.	Verificar quantas vezes um método de Analytics foi invocado e com quais argumentos.
Mock	Objeto pré-programado com expectativas de chamadas que verifica interações estritamente.	Verificar chamadas de contratos de alto nível (ex: Mockito / MockK / Cuckoo).
Fake	Possui uma implementação funcional simplificada (ex: banco de dados em memória).	Testes de integração rápidos sem necessidade de mocks frágeis.

3. IMPLEMENTAÇÃO PRÁTICA: FAKE REPOSITORY

KOTLIN FAKE

```
class FakeUserRepository : UserRepository {
  private val users = mutableListOf<User>()

  override async fun getUser(id: String): User
{
    return users.find { it.id == id }
      ?: throw UserNotFoundException()
  }

  fun saveForTest(user: User) {
    users.add(user)
  }
}
```

SWIFT FAKE

```
final class FakeUserRepository:
UserRepositoryProtocol {
  private var users: [User] = []

  func fetchUser(id: String) async throws ->
User {
    guard let user = users.first(where: { $0.id
== id }) else {
      throw UserError.notFound
    }
    return user
  }
}
```

4. MOCKING DE DEPENDÊNCIAS EM TESTES UNITÁRIOS

MOCKK (KOTLIN)

```
@Test
fun `loadUser emits Success state`() = runTest
{
  val mockRepo = mockk<UserRepository>()
  coEvery { mockRepo.getUser("1") } returns
User("1", "Ana")

  val viewModel = UserViewModel(mockRepo)
  viewModel.load("1")

  assertEquals("Ana",
viewModel.state.value.name)
  coVerify(exactly = 1) { mockRepo.getUser("1")
}
}
```

SWIFT (XCTEST + MANUAL STUB)

```
func testLoadUserSuccess() async throws {
  let stubRepo = StubUserRepository()
  stubRepo.resultToReturn = .success(User(id:
"1", name: "Ana"))

  let viewModel = UserViewModel(repository:
stubRepo)
  await viewModel.load(id: "1")

  XCTAssertEqual(viewModel.state.name, "Ana")
  XCTAssertEqual(stubRepo.fetchCallCount, 1)
}
```